

HONEYWELL SOFTWARE ENGINEERING TEAMS

AI Champions Programme

Module 05

Software Development — SDD to Complete Software SDLC

Taking Module 04's specification through a full greenfield SDLC, then applying the same SDD discipline to safely evolve an existing brownfield application — architecture through release, twice over.

	DURATION		FORMAT		DELIVERED FOR
	4 Hours		Extended Workshop		Honeywell Eng. Teams

How We'll Spend These Four Hours

Six connected moves — two full workflows, greenfield and brownfield, built on the same discipline.

01

Greenfield SDLC: Architecture, Conventions & Release Readiness

35 MIN

How Copilot assists across the full SDLC, from architecture to CI/release, building on Module 04's specification.

02

Greenfield Hands-On: Spec → Release-Ready Feature

60 MIN

Take Module 04's specification through implementation, tests, CI/quality checks, and PR/release readiness.

03

Brownfield: Repository Archaeology & Change-Impact Analysis

35 MIN

Analyzing an existing application's architecture, dependencies, and patterns before touching any code.

04

Brownfield Hands-On: Implementing Without Disruption

60 MIN

Implement a feature in existing code using the minimal-change strategy, then verify existing behavior is preserved.

05

Greenfield vs Brownfield: Comparing the Two Workflows

20 MIN

Where the two workflows start from the same discipline, and where they genuinely diverge.

06

Module Wrap-Up & Debrief

30 MIN

Consolidating what both tracks produced and previewing how Module 06's test strategy builds on both.

Module 05 in the 12-Module Journey

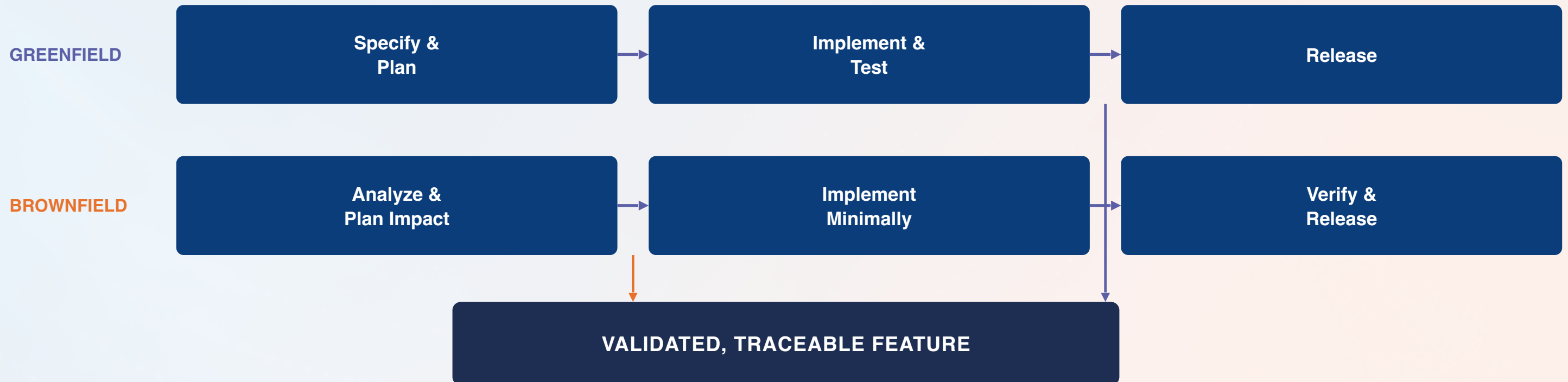
The longest module in the programme — where SDD stops being a technique and becomes a full workflow.



Why this matters: Module 06 derives its test strategy directly from what gets built here; every module from here on assumes you can move comfortably between greenfield and brownfield work.

Two Tracks, One Discipline

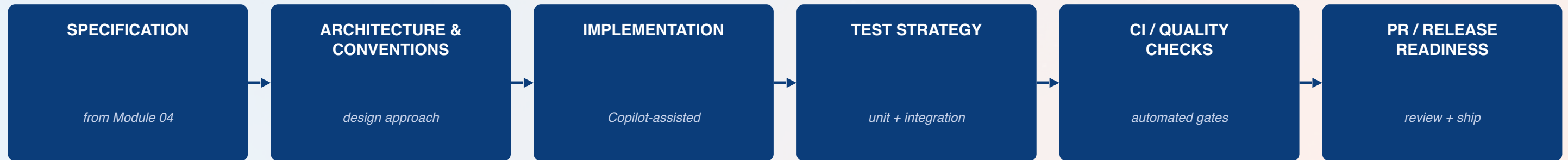
Every stage of the SDLC, walked twice — once building new, once evolving existing code.



Why this matters: Both tracks are built on the same SDD foundation from Module 04 — the difference is what has to be true before you can safely start implementing.

The Greenfield SDLC Flow

Module 04's spec, carried all the way to a release-ready feature.



The throughline: Copilot assists at every stage — not just implementation — which is what turns "AI wrote some code" into "AI helped ship a validated feature."

Copilot Across the Full SDLC, Not Just Code Generation

Four places Copilot contributes beyond writing the implementation itself.



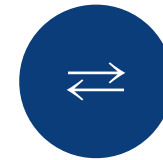
Architecture & Planning

Drafting plan.md options, surfacing trade-offs, and checking the approach against the spec's constraints.



Test Generation

Generating unit and integration tests directly from the specification's acceptance criteria.



CI/CD & Quality Gates

Drafting pipeline configuration and explaining quality-gate failures with the surrounding context.



PR & Release Readiness

Writing PR descriptions, summarizing changes, and checking release readiness against the original spec.

Greenfield Hands-On Lab Preview

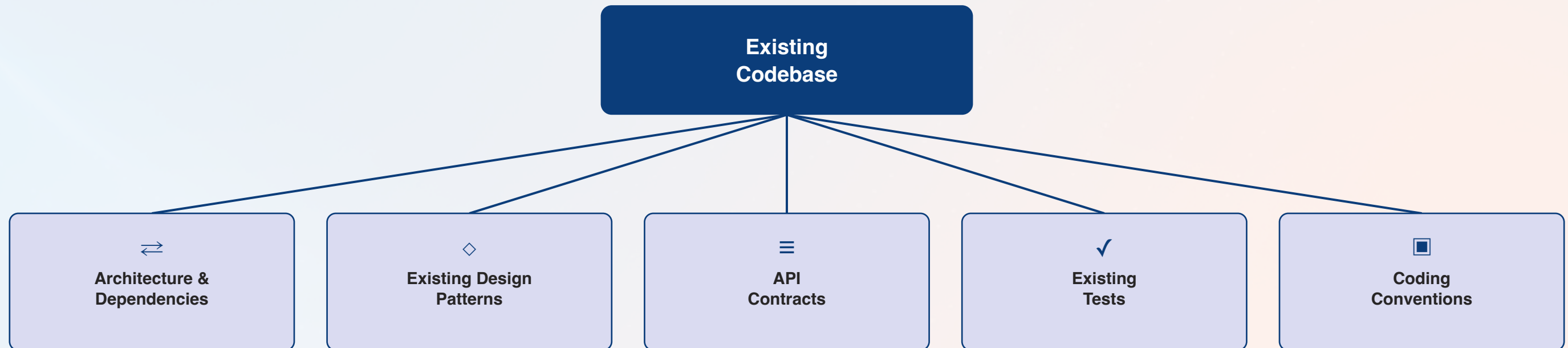
The 60-minute activity that carries Module 04's specification to a release-ready feature.

Stage	What You'll Produce	Validated Against
Architecture / Plan	plan.md	Requirements & constraints
Implementation	Working code, task by task	tasks.md
Test Strategy	Unit + integration tests	Acceptance criteria
CI / Release	Passing pipeline, PR ready to review	The full specification

Every row is checked against Module 04's specification before the feature is considered release-ready.

Repository Archaeology: Before You Touch Brownfield Code

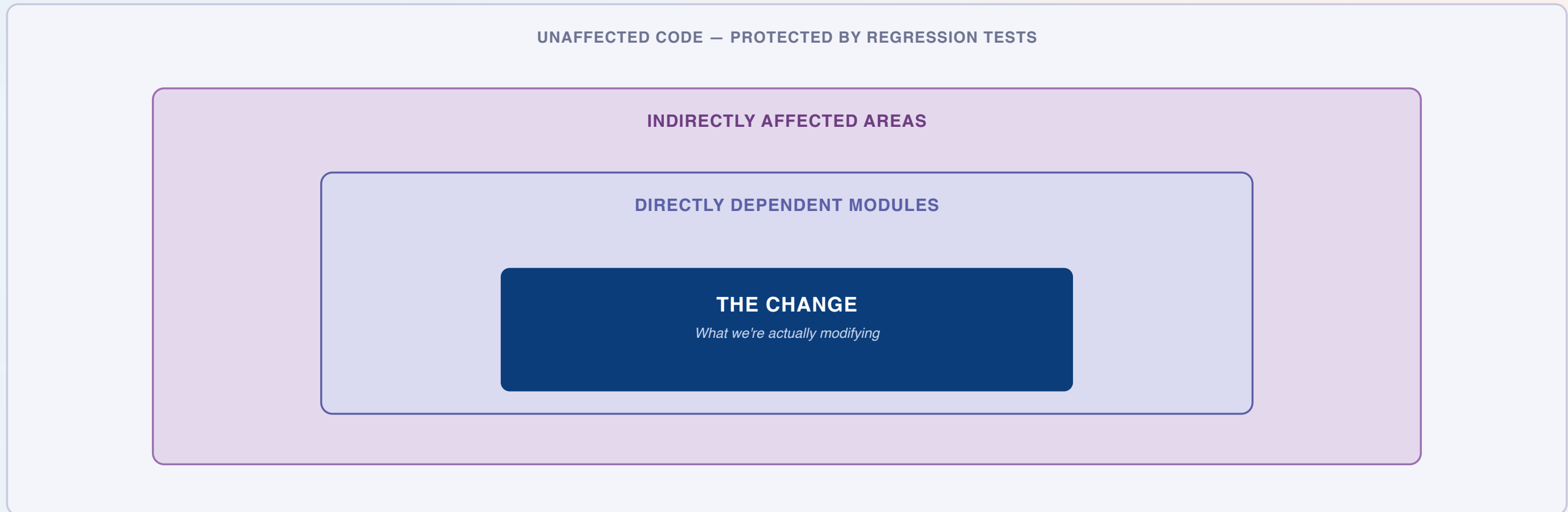
Five things worth understanding before the first line of a brownfield change is written.



Why this matters: Every one of these shapes what "minimal change" means for this specific codebase — skipping this step is how well-intentioned changes become regressions.

Change-Impact Analysis: Mapping the Blast Radius

Before writing code, know exactly how far a change could reach.



Why this matters: The blast radius, not the file you're editing, defines the scope of "minimal change" — and the outer ring is exactly what regression testing exists to protect.

The Minimal-Change Strategy

Once the blast radius is known, discipline is what keeps it small.

MINIMAL-CHANGE STRATEGY

- Change only what the feature actually requires
- Stay within the identified blast radius
- Follow the codebase's existing patterns, even if imperfect
- Re-run the full existing test suite before and after

MAXIMAL DISRUPTION

- Refactor broadly while implementing the feature
- Touch files outside the change's actual scope
- Introduce new patterns alongside existing ones
- Skip re-running the existing test suite

The Brownfield Safe-Evolution Flow

The same rigor as the greenfield flow, shaped by what already exists.



The throughline: Every brownfield stage exists to answer one question before code changes: what could this break, and how do we know it didn't?

Brownfield Hands-On Lab Preview

The 60-minute activity that implements a feature in existing code without unnecessary disruption.

Stage	What You'll Produce	Validated Against
Repository Archaeology	Notes on existing architecture & patterns	The codebase itself
Change-Impact Analysis	A documented change plan	The identified blast radius
Minimal-Change Implementation	The feature, implemented in scope	The change plan
Regression Verification	Full existing test suite, passing	Pre-change behavior, preserved

Success here means the feature works AND every pre-existing test still passes.

Greenfield vs Brownfield: Where the Workflows Diverge

Same discipline, different starting constraints.

Dimension	Greenfield	Brownfield
Starting point	A blank slate, guided by the spec	An existing codebase, guided by the spec + existing patterns
First step	Architecture & conventions	Repository archaeology
Primary risk	Under-specifying requirements	Unintended regression
Change scope	Bounded by the spec	Bounded by the spec AND the blast radius
Validation	Acceptance criteria	Acceptance criteria + full regression suite

Module 05 Outcomes & What's Next

- ✓ The greenfield SDLC flow — spec to release-ready feature — is understood
- ✓ Where Copilot assists across the full SDLC, not just code generation, is clear
- ✓ Repository archaeology and change-impact analysis are practiced on real brownfield code
- ✓ The minimal-change strategy and its do/don't boundaries are understood
- ✓ A feature has been implemented and validated in both a greenfield and a brownfield context
- ✓ The two workflows have been compared side by side



NEXT MODULE

Module 06 — Test Strategy, Testcontainers and End-to-End Validation (2 hours). Deriving a test strategy from the SDD feature, using Testcontainers for realistic service/database validation, and proving the strategy catches a deliberately injected defect.

Questions & Discussion

Module 05 — Software Development — SDD to Complete Software SDLC